Python Activity-Based Learning

Case Study 42: Student Marks Matrix Analysis

1. Problem Statement

Analyse a matrix of student marks across subjects. Calculate each student's total and average, each subject's average, identify the top performer, and flag subjects with low average performance.

2. Objectives

- Store marks in a nested list.
- Use nested loops for row-wise and column-wise calculations.
- Identify maximum values and compare averages with a threshold.

3. Assumptions

- All marks are between 0 and 100.
- Each student has marks for the same number of subjects.
- A subject average below 50 is considered low.

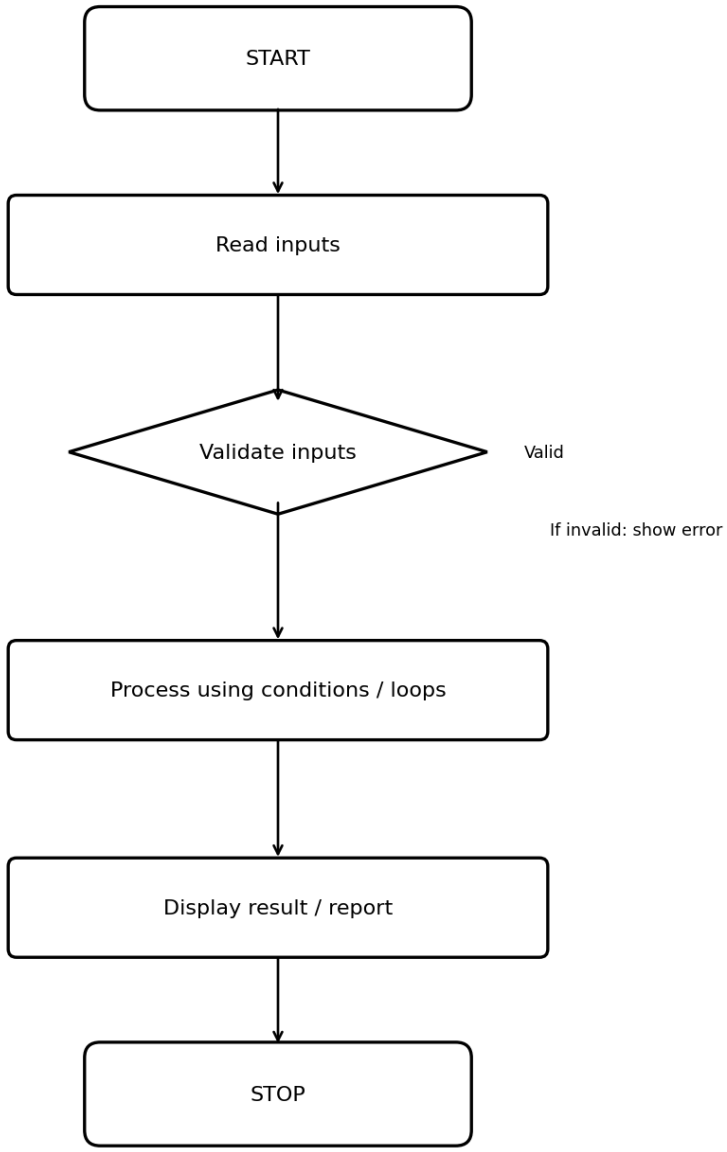
4. Algorithm

1. Read student names, subject names, and marks.
2. Store marks in a nested list and validate each mark.
3. Calculate each student's total and average.
4. Calculate each subject's average by iterating through rows.
5. Find the student with the highest total and flag low-average subjects.
6. Display the report.

5. Flowchart

The diagram below shows the overall program flow. The validation decision represents the input checks described in the algorithm.

Flowchart: Student Marks Matrix Analysis



6. Pseudocode

```
START
READ student_count and subject_count
READ subject names
FOR each student
    READ name and marks for every subject
    STORE marks in nested list
END FOR
FOR each student row
    total = sum of row
    average = total / subject_count
    DISPLAY total and average
END FOR
FOR each subject column
    calculate column average
```

```
    IF average < 50 THEN mark as low-performing
END FOR
FIND student with highest total
DISPLAY report
STOP
```

7. Python Source Code

The solution uses only basic Python features from the assigned syllabus.

```
students = []
subject_count = int(input("Number of subjects: "))
student_count = int(input("Number of students: "))
subjects = []

for j in range(subject_count):
    subjects.append(input(f"Subject {j + 1} name: "))

for i in range(student_count):
    name = input(f"Student {i + 1} name: ")
    marks = []
    for subject in subjects:
        mark = float(input(f"{subject} marks for {name} (0-100): "))
        if mark < 0 or mark > 100:
            print("Invalid mark; using 0 for this entry.")
            mark = 0
        marks.append(mark)
    students.append([name, marks])

print("\nSTUDENT SUMMARY")
top_name = ""
top_total = -1
for name, marks in students:
    total = sum(marks)
    average = total / subject_count
    print(f"{name}: total={total:.1f}, average={average:.1f}")
    if total > top_total:
        top_total = total
        top_name = name

print("\nSUBJECT AVERAGES")
for j in range(subject_count):
    total = 0
    for student in students:
        total += student[1][j]
    average = total / student_count
    status = "LOW AVERAGE" if average < 50 else "OK"
    print(f"{subjects[j]}: {average:.1f} ({status})")

if top_name:
    print(f"Top performer: {top_name} ({top_total:.1f} marks)")
```

8. Test Cases

Test input / condition	Expected result	Purpose
Two students; Maths, Python; marks 80,70 and 60,90	Student averages 75 and 75	Subject averages 70 and 80; tie top performer uses first student
Mark 110	Out of range	Entry set to 0

Subject average 40	Below 50	Flagged LOW AVERAGE

9. Sample Output Screenshot

Sample console output for one test run:

```
C:\Users\Badri\PycharmProjects\PythonProject-1\.venv\Scripts\python.exe "C:\Users\Badri\PycharmProjects\PythonProject-1\Case Studies\Case
 Study - 2.py"
Number of subjects: 2
Number of students: 2
Subject 1 name: Maths
Subject 2 name: Python
Student 1 name: Swaroop
Maths marks for Swaroop (0-100): 80
Python marks for Swaroop (0-100): 90
Student 2 name: Bobby
Maths marks for Bobby (0-100): 70
Python marks for Bobby (0-100): 100

STUDENT SUMMARY
Swaroop: total=170.0, average=85.0
Bobby: total=170.0, average=85.0

SUBJECT AVERAGES
Maths: 75.0 (OK)
Python: 95.0 (OK)
Top performer: Swaroop (170.0 marks)
```

10. Conclusion

The nested-list solution calculates row and column statistics and demonstrates how matrices can represent student marks.

Note: Assumed rates, thresholds, and sample values are for this educational exercise and can be changed if the instructor supplies different rules.